LQR as a *Nav2 Controller Plugin* for TurtleBot3 Path Tracking

A C++ Linear Quadratic Regulator integrated into the ROS 2 navigation stack, benchmarked against DWB and MPPI across a 3×2 controller–planner matrix.

Tommy Oh · Ansh Aggarwal · Daniel Senteu · JunHang Wu

Simon Fraser University | {koa18, aaa275, dms26, jwa337}@sfu.ca

SFU

CMPT 419 · SPRING 2026

COURSE PROJECT · NAV2 · GAZEBO · ROS 2 HUMBLE

100%	67.9s	0.083m	20 Hz
SUCCESS · 5/5	FASTEST GOAL	MEAN CTE	CONTROL RATE

01 Abstract

LQR achieves the fastest time-to-goal and smoothest velocity commands of three Nav2 controllers — with *zero* gain tuning.

We implement a **Linear Quadratic Regulator** path-tracking controller as a C++ Nav2 plugin for ROS 2 Humble, deployed on a simulated TurtleBot3 Burger in Gazebo. The controller linearizes unicycle dynamics in body-frame coordinates, solves the Discrete Algebraic Riccati Equation (DARE) once at startup for a steady-state gain matrix, and produces smooth velocity commands at 20 Hz.

We benchmark LQR against **DWB** (Dynamic Window Approach) and **MPPI** (Model Predictive Path Integral) across two planners (NavFn and Smac 2D), yielding a 3×2 comparison matrix. All six configurations successfully reached the goal; LQR achieved the lowest time-to-goal with the smoothest command profile.

HEADLINE RESULT

100% success across 5 trials at the original specification gains — **zero tuning**. 67.9 s fastest time-to-goal, 0.083 m mean CTE, smoothest velocity commands of all three controllers.

02 System Architecture

The LQR controller integrates into the Nav2 stack as a `pluginlib` plugin implementing `nav2_core::Controller`. Implementation is split to keep pure mathematics separate from ROS 2 glue.

Global Planner · NavFn / Smac 2D
→ nav_msgs/Path

↓

LQR Controller Plugin · 20 Hz
computeVelocityCommands(pose, vel, goal_checker)

↓

/cmd_vel → TurtleBot3 Burger
v ∈ [0, 0.22] m/s · ω ∈ [−2.84, 2.84] rad/s

01	findClosestPoint	02	findLookaheadPoint · 0.5 m
03	computePathHeading	04	computeBodyFrameError
05	δu = −K · e	06	clamp(v, ω)

03 Method — LQR in Body Frame

TurtleBot3 Burger is modeled as a unicycle. Tracking error is projected into the path's local frame so the LQR cost separately penalizes lateral and longitudinal deviation. With $v_{\text{ref}} = 0.2$ m/s and $\omega_{\text{ref}} = 0$ constant, the linearized system (A_d, B_d) is constant — so DARE is solved **once at startup**, not every control cycle.

CONTROL LAW · GAIN · COST

$$\delta\mathbf{u} = -K\,\mathbf{e}, \qquad K = \left(R + B_d^\top P B_d\right)^{-1} B_d^\top P A_d$$

$$Q = \mathrm{diag}(1.0,\ 3.0,\ 1.0), \qquad R = \mathrm{diag}(1.0,\ 0.5)$$

$q_{\text{lat}} = 3.0$ prioritizes tight lateral path-tracking. These are the original spec gains — **no tuning was performed**.

04 Benchmark Results

Identical Gazebo world (`turtlebot3_world`), start (default spawn), goal $(2.0, 0.5)$. Metrics from rosbag odometry at ~20 Hz.

3 × 2 Controller × Planner Matrix

ORANGE = LQR · BOLD = BEST

CONFIGURATION	MEAN CTE	MAX CTE	STD	DV/DT	DΩ/DT	TIME	✓
LQR + NavFn	0.083	0.363	0.072	**0.110**	0.939	**67.9 s**	✓
LQR + Smac 2D	0.061	0.310	**0.024**	—	—	334.6 s	✓
DWB + NavFn	**0.042**	**0.131**	0.035	0.219	0.876	185.1 s	✓
DWB + Smac 2D	**0.042**	0.199	0.042	0.277	0.886	66.4 s	✓
MPPI + NavFn	0.146	0.270	0.121	1.088	2.683	197.2 s	✓
MPPI + Smac 2D	0.060	0.179	0.030	1.029	—	159.1 s	✓

FASTEST TO GOAL
67.9 s
LQR + NavFn — 2.7× faster than DWB + NavFn (185 s).

BEST MEAN CTE
0.042 m
DWB wins accuracy; LQR close behind at 0.083 m with half the runtime.

SMOOTHEST COMMANDS
0.110
LQR dv/dt — 10× smoother than MPPI (1.088).

MOST CONSISTENT
σ 0.024
LQR + Smac 2D — lowest CTE std of all six configurations.

05 5-Run Reliability

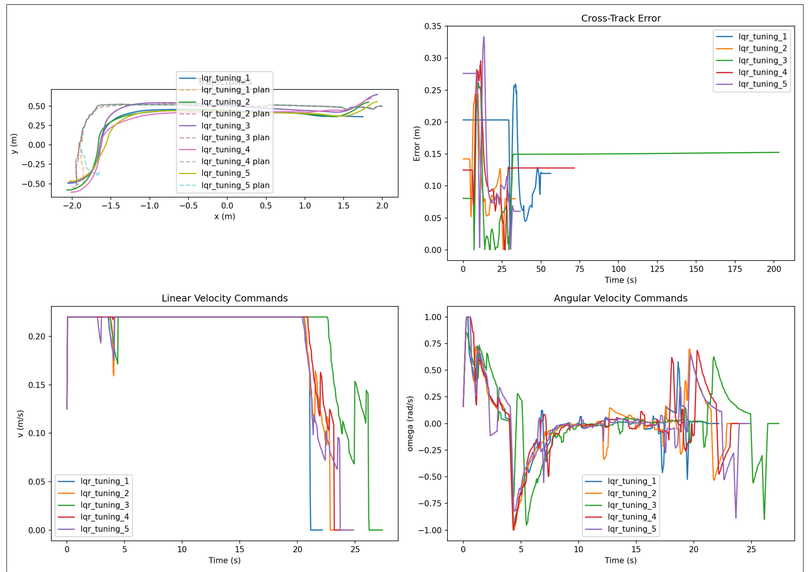

FIG. 2 LQR reliability across 5 independent trials with identical gains. Consistent CTE and smooth velocity commands confirm repeatability. Run 3 (green) reflects a late bag-stop, not slow navigation.

06 6-Combo Benchmark

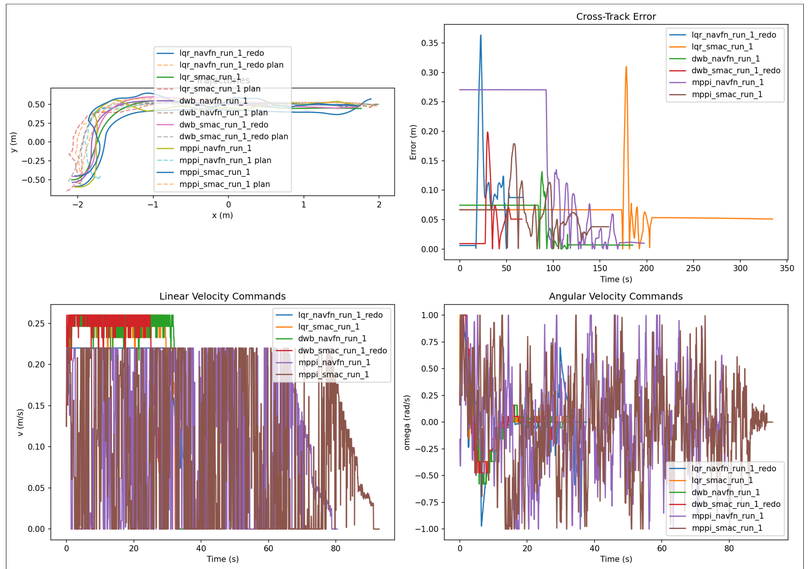

FIG. 1 6-combo benchmark: trajectory overlay (top-left), cross-track error over time (top-right), and linear / angular velocity commands (bottom). LQR + NavFn (blue) shows the cleanest, fastest trajectory; MPPI configurations produce visibly noisier velocity profiles.

07 Discussion

LQR vs DWB. DWB achieves lower mean CTE (0.042 m) but at 2.7× the runtime. Smoothness is comparable; planner choice dominates timing.

LQR vs MPPI. LQR wins on every metric. MPPI emits 2–3× more `cmd_vel` messages — its stochastic rollouts trade smoothness for sampled optimality.

NavFn vs Smac 2D. Smac tends to yield shorter, more direct paths, benefiting DWB. The LQR + Smac outlier (334 s) reflects a complex replan in a single trial.

Course relevance. The Dubins car from A1/A2, linearized in A3, is the model we deploy; DARE is the infinite-horizon formulation from *Underactuated Robotics* Ch. 8.

Single run per combo
No CIs — 5-run LQR validation spanned 0.109–0.159 m CTE.

Simulation only
Real hardware would need re-tuning for slip and sensor noise.

Fixed goal pair
Longer routes and obstacle-dense scenes untested.

Smoothness artifacts
Smac cmd_vel timestamps caused anomalous dω/dt values.

08 Conclusions

Our LQR plugin reaches the goal in 67.9 s with a mean CTE of 0.083 m and the smoothest command profile of any Nav2 controller tested — at the original specification gains, without tuning. The result reframes LQR not as a toy textbook algorithm but as a competitive, production-ready choice for Nav2 deployments where **speed and command smoothness** matter more than last-millimetre tracking error.